

TUGAS 2 - SISTEM PARALLEL DAN TERDISTRIBUSI

Sinkronisasi dan Distributed Systems

Identitas Tugas

- **Mata Kuliah:** Sistem Parallel dan Terdistribusi
- **Judul Tugas:** Implementasi Distributed Synchronization System
- **Jenis:** Tugas Individu
- **Estimasi Waktu:** 5 Jam Kerja
- **Deadline:** 3 Mei 2026
- **Bobot Nilai:** 30% dari total nilai akhir

DESKRIPSI TUGAS

Kembangkan sistem sinkronisasi terdistribusi yang mensimulasikan skenario real-world dari distributed systems. Sistem ini harus mampu menangani multiple nodes yang berkomunikasi dan mensinkronisasi data secara konsisten.

SPESIFIKASI TEKNIS

1. Core Requirements (Wajib - 70 poin)

A. Distributed Lock Manager (25 poin)

- Implementasi distributed lock menggunakan algoritma Raft Consensus
- Minimum 3 nodes yang dapat saling berkomunikasi
- Support untuk shared dan exclusive locks
- Handle network partition scenarios
- Implementasi deadlock detection untuk distributed environment

B. Distributed Queue System (20 poin)

- Implementasi distributed queue menggunakan consistent hashing
- Support untuk multiple producers dan consumers
- Implementasi message persistence dan recovery
- Handle node failure tanpa kehilangan data
- Support untuk at-least-once delivery guarantee

C. Distributed Cache Coherence (15 poin)

- Implementasi cache coherence protocol (pilih salah satu: MESI/MOSI/MOESI)
- Support untuk multiple cache nodes
- Handle cache invalidation dan update propagation
- Implementasi cache replacement policy (LRU/LFU)
- Performance monitoring dan metrics collection

D. Containerization (10 poin)

- Buat Dockerfile untuk setiap komponen
- Implementasi docker-compose untuk orchestration (jika memungkinkan)
- Support untuk scaling nodes secara dinamis
- Environment configuration menggunakan .env files

2. Documentation & Reporting (Wajib - 20 poin)

A. Technical Documentation (10 poin)

- Arsitektur sistem lengkap dengan diagram
- Penjelasan algoritma yang digunakan
- API documentation dengan OpenAPI/Swagger spec
- Deployment guide dan troubleshooting

B. Performance Analysis Report (10 poin)

- Benchmarking hasil dengan berbagai skenario
- Analisis throughput, latency, dan scalability
- Comparison antara single-node vs distributed
- Grafik dan visualisasi performa

3. Video Demonstration (Wajib - 10 poin)

Video YouTube Publik (10 poin)

- Durasi minimal 10 menit, maksimal 15 menit
- Bahasa Indonesia yang jelas dan profesional
- Struktur video:
 - Pendahuluan dan tujuan (1-2 menit)
 - Penjelasan arsitektur sistem (2-3 menit)
 - Live demo semua fitur (5-7 menit)

- Performance testing (2-3 menit)
- Kesimpulan dan tantangan (1-2 menit)
- Video harus publik dan accessible
- Sertakan link video dalam laporan

BONUS FEATURES (Opsional - Maks 15 poin tambahan)

- **Pilihan A: Advanced Consensus Algorithm (5-10 poin)**
 - Implementasi PBFT (Practical Byzantine Fault Tolerance)
 - Handle Byzantine failures (up to $f = (n-1)/3$ faulty nodes)
 - Demonstrasi ketahanan terhadap malicious nodes
 - +5 poin untuk implementasi dasar, +10 poin untuk complete implementation
- **Pilihan B: Geo-Distributed System (5 poin)**
 - Simulasi multi-region deployment
 - Implementasi latency-aware routing
 - Support untuk eventual consistency model
 - Demonstrasi data replication antar region
- **Pilihan C: Machine Learning Integration (5 poin)**
 - Implementasi adaptive load balancing menggunakan ML
 - Predictive scaling berdasarkan traffic patterns
 - Anomaly detection untuk system failures
 - Performance optimization recommendations
- **Pilihan D: Security & Encryption (5 poin)**
 - End-to-end encryption untuk inter-node communication
 - Implementasi RBAC (Role-Based Access Control)
 - Audit logging dan tamper-proof logs
 - Certificate management untuk node authentication

STACK TEKNOLOGI

Wajib digunakan:

- Python 3.8+ (asyncio recommended)
- Docker & Docker Compose (jika memungkinkan)
- Redis (untuk distributed state)
- Network libraries: asyncio, aiohttp, or zeromq
- Testing: pytest, locust (untuk load testing)

Optional tools:

- gRPC untuk inter-node communication
- Prometheus & Grafana untuk monitoring
- Kubernetes untuk advanced orchestration
- Apache Kafka untuk message streaming

PROJECT STRUCTURE

```
distributed-sync-system/  
├── src/  
│   ├── nodes/  
│   │   ├── __init__.py  
│   │   ├── base_node.py  
│   │   ├── lock_manager.py  
│   │   ├── queue_node.py  
│   │   └── cache_node.py  
│   ├── consensus/  
│   │   ├── __init__.py  
│   │   ├── raft.py  
│   │   └── pbft.py (optional)  
│   ├── communication/  
│   │   ├── __init__.py  
│   │   ├── message_passing.py  
│   │   └── failure_detector.py  
│   └── utils/  
│       ├── __init__.py  
│       ├── config.py  
│       └── metrics.py  
├── tests/  
│   ├── unit/  
│   ├── integration/  
│   └── performance/  
├── docker/  
│   ├── Dockerfile.node  
│   └── docker-compose.yml  
├── docs/  
│   ├── architecture.md  
│   ├── api_spec.yaml  
│   └── deployment_guide.md  
├── benchmarks/  
│   └── load_test_scenarios.py  
├── requirements.txt  
├── .env.example  
├── README.md  
└── report.pdf
```

RUBRIK PENILAIAN

Functionality (70 poin)

Kriteria	Bobot	Excellent (90-100%)	Good (70-89%)	Fair (50-69%)	Poor (<50%)
Distributed Lock Manager	25	Raft consensus sempurna, handle all failure scenarios	Fungsional dengan minor issues	Banyak bugs tapi konsep benar	Tidak fungsional
Distributed Queue	20	Semua requirement terpenuhi, excellent error handling	Fungsional dengan sedikit bugs	Banyak bugs tapi basic concept works	Tidak fungsional
Cache Coherence	15	Protocol implementation perfect, great performance	Fungsional dengan minor issues	Banyak bugs tapi konsep benar	Tidak fungsional
Containerization	10	Perfect Docker setup, easy scaling	Good containerization	Banyak issues tapi bisa jalan	Tidak bisa di-containerize

Documentation & Analysis (20 poin)

Kriteria	Bobot	Excellent (90-100%)	Good (70-89%)	Fair (50-69%)	Poor (<50%)
Technical Documentat ion	10	Complete, professional, easy to understand	Good documentati on dengan minor gaps	Cukup tapi banyak missing info	Kurang atau tidak ada
Performanc e Analysis	10	Comprehen sive analysis dengan excellent visualisasi	Good analysis dengan proper metrics	Basic analysis, kurang depth	Minimal atau tidak ada

Video Presentation (10 poin)

Kriteria	Bobot	Excellent (90-100%)	Good (70-89%)	Fair (50-69%)	Poor (<50%)
Content Quality	5	Professional, comprehens ive, engaging	Good presentation dengan minor issues	Cukup tapi kurang structure	Poor quality atau tidak sesuai
Technical Demonstrati on	5	Perfect demo semua features	Good demo dengan minor issues	Demo kurang lengkap	Tidak bisa demonstrat e

Bonus Features (Maks 15 poin tambahan)

- PBFT Implementation: +10 untuk complete, +5 untuk partial
- Geo-Distributed: +5 untuk complete implementation
- ML Integration: +5 untuk innovative solution
- Security Features: +5 untuk comprehensive security

PENGUMPULAN

Format Pengumpulan:

- **Source Code:** Upload ke GitHub (public repository)
- **Documentation:** PDF format dengan nama report_[NIM]_[Nama].pdf
- **Video:** Link YouTube dicantumkan di README dan report
- **Docker Hub:** Push images ke Docker Hub (opsional tapi direkomendasikan)

File yang harus dikumpulkan:

- Link GitHub repository
- Link YouTube video (publik)
- PDF report via [platform pengumpulan]
- Screenshots hasil testing
- File .env (tanpa sensitive data)

DEADLINE & POLICY

- **Deadline:** 3 Mei 2026 pukul 18:00 WITA
- **Late submission:** -10% per hari keterlambatan
- **Plagiarism:** Nilai 0 untuk semua yang terlibat
- **Incomplete submission:** Penilaian hanya untuk yang dikumpulkan

RESOURCES & REFERENCES

Wajib Dibaca:

- Raft Consensus Algorithm Paper
- Distributed Systems: Principles and Paradigms
- Redis Distributed Lock Documentation

Optional References:

- PBFT Paper
- Distributed Systems for Fun and Profit
- Designing Data-Intensive Applications

Video Tutorials:

- Raft Consensus Algorithm Visualization

- Distributed Systems MIT Course
- Docker Networking for Distributed Systems

TIPS SUKSES

- **Start Early:** Distributed systems complex, butuh waktu untuk debugging
- **Test Incrementally:** Test setiap komponen sebelum integrate
- **Use Logging:** Implement comprehensive logging untuk debugging
- **Monitor Performance:** Gunakan metrics untuk identify bottlenecks
- **Handle Failures:** Design for failure dari awal
- **Document As You Go:** Jangan tunggu akhir project
- **Practice Demo:** Rehearse video presentation beberapa kali
- **Seek Help:** Gunakan office hours dan forum diskusi

FAQ

- **Q: Apakah harus semua bonus features dikerjakan?**
A: Tidak, pilih yang Anda kuasai dan sesuai minat. Kualitas lebih penting daripada kuantitas.
- **Q: Bagaimana jika ada network issues selama demo?**
A: Siapkan offline demo sebagai backup, dan jelaskan network challenges dalam video.
- **Q: Apakah boleh menggunakan library existing untuk consensus?**
A: Untuk core requirements, implementasi harus dari scratch. Untuk bonus, boleh menggunakan existing libraries.
- **Q: Minimal berapa nodes yang harus berjalan simultaneously?**
A: Minimal 3 nodes untuk demonstrasi distributed behavior, tapi design untuk support lebih banyak.